

AsanaAI: Project Report

Design, Implementation, Deployment and Verification of a Privacy-Preserving
Yoga Posture Coach on Commodity Hardware

Anamitra Sarkar

Debjit Deb Barman
Tanima Samanta

Adrija Ray Mondal

Supervisor: Mr. Sujit Chakraborty, Assistant Professor, Dept. of CSE (AI/ML)
RCC Institute of Information Technology, Kolkata, West Bengal, India

Contents

1	Introduction	3
2	Related Work	4
3	System Overview	4
4	Data and Labels	5
4.1	Video corpus and cue-verified labels	5
4.2	Features, and a train/serve mismatch	5
4.3	Photo sets	6
4.4	Evaluation protocol	6
5	Models	6
5.1	3-head residual MLP	6
5.2	The cascade: one model names, another vetoes	7
5.3	Rule engine and per-joint cues	7
5.4	ST-GCN “flow check”	7
6	Occlusion, Personalisation and Safe Coaching	7
6.1	Visibility-aware scoring	7
6.2	Would mesh recovery do better? CLIFF versus the mirror	8
6.3	Personal range-of-motion calibration	8
6.4	Safe, multilingual coaching	9
7	Deployment on Real Phones	9
8	Results	10
8.1	Naming the pose from one frame	10
8.2	Baselines: is the neural network necessary?	11
8.3	Held-out videos: the ST-GCN	12
8.4	Does the form score mean anything?	13
8.5	End-to-end checks on a real phone and on recorded video	14
8.6	The interface	14
9	Previous Framework Versus the Current Framework	14
9.1	The MLP, generation by generation	15
9.2	Is the sequence model a genuine ST-GCN?	16

9.3 The sequence model on movement	17
10 Comparison with Related Work	18
11 Limitations and Future Work	19
12 Conclusion	20
13 Verification, Audit and Fixes	21
13.1 Method	21
13.2 Findings that affect users or safety	21
13.3 Hygiene findings	21
13.4 Verified without findings	21
13.5 Calibration check	22
13.6 A measurement worth recording	22
14 Repository, Operations and Reproducibility	22
14.1 Layout	22
14.2 Deployment and operations	22
14.3 Testing	22
14.4 Compute policy	22
14.5 Rebuilding the documents	23

Abstract

Practising yoga without an instructor can risk injury, yet automated coaches are usually validated on random train/validation splits that overstate how well they work for new people, rooms and cameras. We present AsanaAI, a browser-based yoga posture coach in which the camera image never leaves the device: the page extracts 33 body landmarks, and only those numbers reach a small CPU server. The server runs a cascade of 3-head residual MLPs (one model names the pose, a second vetoes photos that are not one of the app’s poses and scores form), a rule engine that turns joint-angle bands into per-joint cues, and an ST-GCN over the 33-joint skeleton as a second opinion on held poses. Training labels come from 24 instructor-led videos (1.2 M frames) and are cue-verified: the instructor names the pose in a Whisper transcript, a biomechanical rule engine agrees, and the pose is held steady. Scoring only on data never used in training, the cascade names the pose on 78.9% of 1,685 held-out public photos with a 20.8% false-alarm rate on photos of other poses and clears a 0.70 recall/precision bar on seven poses; the rules-only path it replaces reaches 36.9%, 78.3% and none. On held-out videos the ST-GCN reaches 78.0% macro recall, against 19–21% for the sequence model it replaces. Against standard methods trained on the same data, a random forest is stronger on held-out photos from the training sources (85.3% vs. 78.9% overall) but weakest on wild photos from another source (35.0% vs. 55.3%), so no single method wins everywhere. Deployment work makes this usable on budget phones: a pose-engine fallback ladder for a GPU driver that crashes MediaPipe, visibility-aware scoring that declines to judge joints the camera cannot see, a personal range-of-motion calibration, an on-device port of the rule engine (0 mismatches over 12,500 parity cases), and English/Hindi/Bengali coaching passed through a safety screen. We report what does not work: no live-person accuracy study yet, weak mountain, cobra and child’s pose, and a form score that is nearly blind to arm errors.

Keywords— yoga posture correction, pose estimation, residual MLP, ST-GCN, occlusion, on-device inference, held-out evaluation

1 Introduction

Yoga is practised worldwide—the United Nations proclaimed an International Day of Yoga in 2014 [1]—and it is increasingly learned alone, from videos and phone apps, with nobody to correct the posture. A systematic review of published case reports and case series documented adverse events associated with yoga [2]. A camera-based coach that tells a practitioner which joint is off, in their own language and without frightening or pushing them, is therefore attractive. Real-time body-landmark estimators that run on a phone [3,4] make it technically possible; making it trustworthy on ordinary hardware is harder, and this paper is about that gap.

We build on three observations from developing and testing AsanaAI.

(1) *Random-split accuracy is not accuracy.* Video frames and web photos have near-duplicate neighbours, mirrored copies and repeated photographs; when these fall on both sides of a random split, validation accuracy measures memory, a form of leakage known to inflate reported results across many fields [5]. The first model of this project reached 91.5% on its random validation split; on held-out photos it passed none of the poses and raised a false alarm on 75% of photos of poses outside its vocabulary (Section 8). We therefore report only numbers measured on data a model never trained on.

(2) *A coach must be deployable where its users are.* Phones differ widely in their GPU drivers. On one budget phone (MediaTek Dimensity 7025, PowerVR BXM-8-256 GPU) we found that the standard MediaPipe web engine aborts inside the driver, and we chose to send no image to a server, because a camera pointed at a person at home should not become an upload.

(3) *Feedback has to be honest about what is not seen.* A pose estimator happily invents joints that are hidden; a coach that scores an invented knee is worse than silent. And a generated sentence must never tell someone to push through pain.

Contributions.

1. *Cue-verified labels.* A labelling protocol for instructor videos—the instructor names the pose in an automatic transcript [6], a rule engine agrees, and the pose is held steady—that turns 1.2 M video frames into ~80 k trustworthy positive frames, and a finding that the original training table was built with 3-D angles while the app serves 2-D angles (median gap 14°).

2. *A held-out evaluation protocol* with an explicit false-alarm metric for photos of poses outside the vocabulary, a fixed pass bar (recall and precision ≥ 0.70), and controlled baselines trained on the same data.
3. *A gated cascade* of 3-head residual MLPs in which one model names the pose and another vetoes “not one of mine”, plus an ST-GCN [7] second opinion whose input rate we show must match training.
4. *Occlusion-aware, personalised, safe coaching*: visibility-aware scoring, a comparison of left–right mirroring with a mesh-recovery model (CLIFF [8]) for hidden joints, a per-user range-of-motion calibration, and a screened, three-language coaching pipeline.
5. *Engineering for the real world*: a pose-engine fallback ladder verified over USB on a real phone whose GPU path fails, an offline rule-engine port with a parity proof, and live screenshots of the deployed interface.

We also report negative results: weak poses, arm-error blindness, and a mesh-recovery model we chose not to ship.

Section 2 reviews related work; Sections 3–5 describe the system, data and models; Sections 6 and 7 cover occlusion, personalisation and deployment; Section 8 gives results; Section 10 compares with prior work; Section 11 states limitations and future work.

2 Related Work

Yoga pose recognition. Early work recognised postures from silhouettes with a star-skeleton descriptor for self-training [9]. Yoga-82 [10] offers 82 pose classes with three-level hierarchical labels and motivates its difficulty by pose diversity, occlusion and viewpoint. Recent studies classify landmarks or skeleton images: Mohiuddin et al. compare MediaPipe and YOLOv8 skeleton inputs with raw images on Yoga-16 (1,280 images, 16 poses) and report 96.09% for a VGG16 on MediaPipe skeletons [11]. Such results come from a split of one small image set, so they say little about photos from other sources or of other poses.

Posture feedback. Thoutam et al. detect incorrect yoga posture in uploaded videos and report abnormal angles with advice [12]. PosePilot pairs an LSTM with a BiLSTM and multi-head attention to provide corrective feedback for edge devices and introduces a yoga video dataset [13]. Pose-to-Pose studies transitions between yoga poses with counterfactual explanations [14]. In rehabilitation, Liao et al. learn to score exercise quality [15]. We borrow the idea of angle-based feedback but add what these works do not report in the sources we read: held-out evaluation with a false-alarm metric, occlusion-aware scoring, privacy by design, multilingual screened coaching and a verified phone deployment (Section 10).

Pose estimation and occlusion. BlazePose [3] within the MediaPipe framework [4] gives 33 landmarks in real time on phones; OpenPose [16], HRNet [17] and ViTPose [18] are more accurate but heavier; TensorFlow.js [19] brings estimators to browsers. Hidden limbs remain hard. Mesh-recovery models fit a body model [20] and, like CLIFF [8], can place hidden joints plausibly, but they need the image. We measure CLIFF against a trivial left–right mirror (Section 6).

Skeleton sequences. ST-GCN [7] applies graph convolution over a skeleton and temporal convolution over time. We use it for held poses over 60 frames and test whether the graph structure matters.

Evaluation and systems hygiene. Leakage inflates ML results [5]; selective classification gives a classifier the option to abstain [21], which is the role of our “not one of mine” veto; and ML systems accumulate hidden technical debt such as silent fallbacks and train/serve skew [22], both of which we hit.

3 System Overview

AsanaAI is a web app (a progressive web app that also installs on a phone’s home screen) with a public landing page and the practice screen. Fig. 1 shows the data flow.

On the device. The browser opens the camera and a pose engine returns 33 landmarks (x, y, z, v) per frame, where v is the estimator’s visibility. Visibility is smoothed with hysteresis so that a joint does not flicker

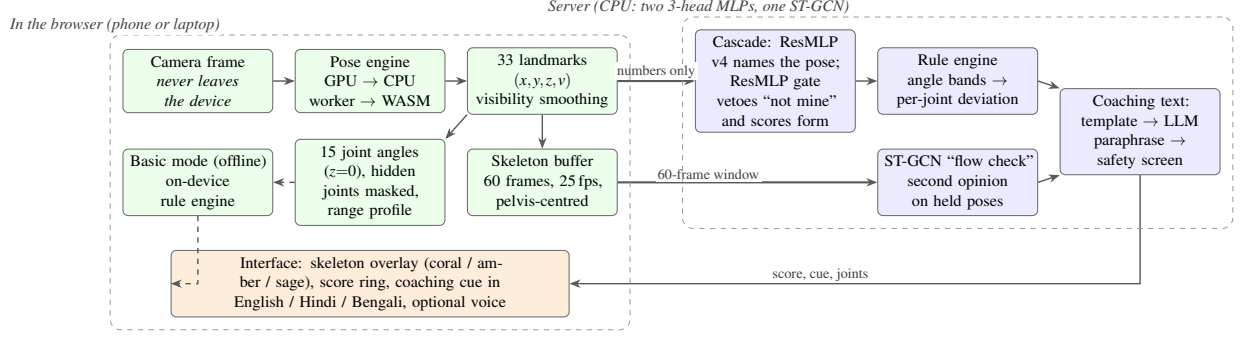


Figure 1: AsanaAI architecture. Everything left of the network arrow runs in the browser: the camera frame is turned into 33 landmarks by a pose engine chosen automatically (Section 7), hidden joints are masked, and only landmark-derived numbers are sent. The server holds three small models and a rule engine; a rule-engine port in the browser keeps a basic mode working offline. Dashed arrows show the offline path.

between seen and hidden. From the landmarks the page computes 15 joint angles (Section 4.2) and, separately, a rolling 60-frame skeleton buffer resampled to 25 fps. The camera frame itself is never uploaded; only numbers are sent: the 15-angle vector (with the user’s calibration ranges, if any) and, for the sequence model, the buffered landmark coordinates, so the privacy promise on the landing page (“your video stays on your device”) is enforced by construction. A per-user calibration profile is kept in the browser’s local storage (Section 6.3).

On the server. A CPU service (a Hugging Face Space) receives the angle vector at about one request every 1.5 s. A cascade of two 3-head residual MLPs names the pose and decides whether it is “not one of mine” (Section 5.2); a rule engine compares the angles with each pose’s reference bands and returns per-joint deviations; an ST-GCN scores 60-frame windows as a second opinion we call the *flow check* (Section 5.4). The coaching sentence is composed from reviewed templates and optionally paraphrased by a language model behind a safety screen (Section 6.4). All three networks plus the rule engine fit in 324 MB peak memory, and server compute is 7–9 ms per request (median).

Two practice modes. In *free mode* nothing is selected and the server names whatever it recognises (the pose guide lists six reference poses); *Guided mode* lets the user pick one of 8 target poses (Warrior II, Tree, Triangle, Chair, Downward Dog, Plank, Seated Easy, Corpse), shows the credited reference photo and measured joints, and replaces the score by a wrong-pose card if the camera sees another pose.

Languages and voice. The interface, cues and optional speech are available in English, Hindi and Bengali.

4 Data and Labels

4.1 Video corpus and cue-verified labels

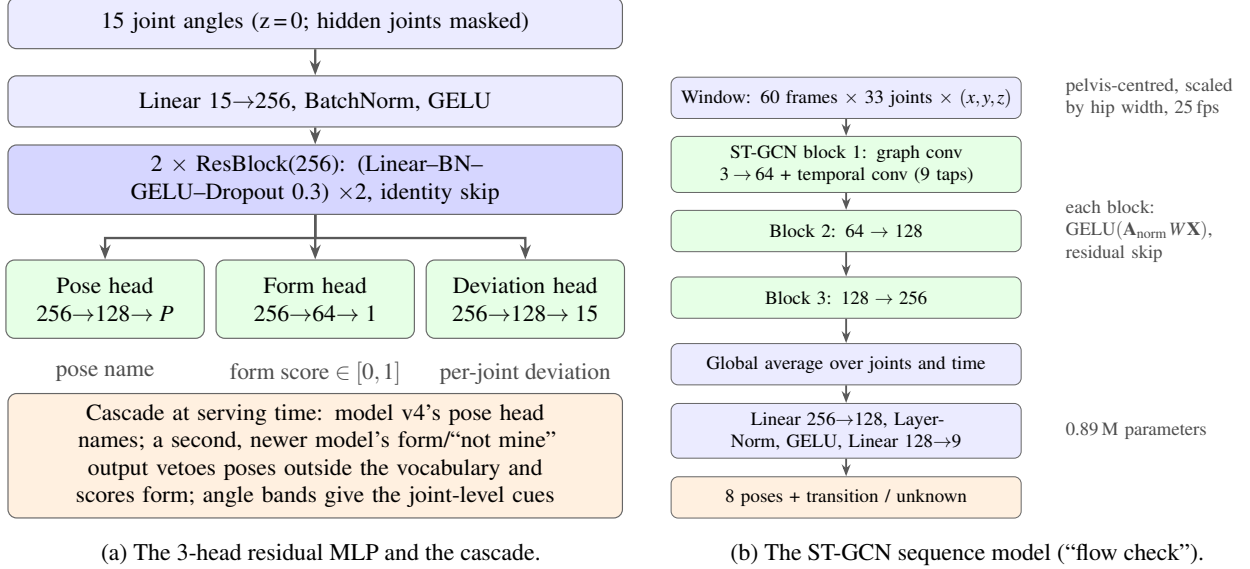
We collected 24 instructor-led yoga videos (12 original and 12 added later), 1,206,391 frames in total, and ran MediaPipe on every frame. Frame labels are *cue-verified*: a frame is a positive example of a pose only if (i) the instructor names that pose in the speech transcript (Whisper-large-v3 [6], English and Hindi), and (ii) the app’s own rule engine agrees with the name, and (iii) the body is held steady. This yields about 80 k positive frames; about 421 k frames are labelled transition (the body is moving between poses) and about 705 k are excluded as ambiguous. The point of the triple check is that no one of the three sources is trusted alone: an instructor may name a pose before or after the body reaches it, and labels from a rule engine alone would only teach a model to copy the rules.

4.2 Features, and a train/serve mismatch

The models see 15 joint angles computed from the 2-D landmarks (the z coordinate is set to zero): left/right elbow, shoulder, hip, knee and ankle angles; left/right trunk angles; the neck angle; and left/right hip abduction. This is exactly what the browser can compute cheaply and what is sent. An audit showed that the original training table had been built with raw 3-D angles while the app serves zero- z angles: the median per-feature gap was 14° , a pure train/serve skew [22]. All training tables used here are rebuilt with the served recipe.

Table 1: Data used in this paper.

Item	Size / detail
Training videos	24, 1,206,391 frames; ~ 80 k positive / ~ 421 k transition / ~ 705 k excluded
Features	15 zero-z joint angles per frame (MLP); $60 \times 33 \times 3$ landmark windows (ST-GCN)
Commons-422	422 photos; frozen test split 103
Public photos	6,616 (4,931 train / 1,685 held out, of which 1,037 are “other poses”)
ST-GCN windows	24 videos, 8,002 windows (60 frames, stride 12), 3 folds by video

**Figure 2:** The two learned models. P is the number of pose classes. For $P \approx 22$ the MLP has about 0.36 M parameters; the nine-class ST-GCN has about 0.89 M (both counted from the model definitions).

4.3 Photo sets

Photos supply the variety that 24 videos cannot. Commons-422 is a set of 422 freely licensed Wikimedia photos with a frozen test split of 103 (hash-split, fixed since 19 Sep 2026). Public photos are 6,616 photos from two public datasets: 2,634 labelled with our poses and 3,982 of poses outside our vocabulary, used as negatives; 4,931 are used for training and 1,685 are held out (1,037 of them are “other poses”). Table 1 summarises the data.

4.4 Evaluation protocol

A number counts only if it was measured on data the model never trained on. A pose *passes* if recall ≥ 0.70 and precision ≥ 0.70 with at least 10 held-out photos (5 on the tiny frozen set; 20 windows for the ST-GCN). A *false alarm* is a photo of a pose that is not one of the app’s poses that the system names as one of them (lower is better). Sequence models are scored on three folds split by video, so the model never sees the person or room it is tested on. The trainers’ own validation accuracy (90–98%) comes from a random split and is never quoted as a result.

5 Models

5.1 3-head residual MLP

The pose model takes the 15-angle vector (Section 4.2) and passes it through a 256-unit input layer (Linear, batch normalisation [23], GELU [24]) and two residual blocks [25] of width 256 (two Linear-BN-GELU-Dropout stages with an identity skip). Three heads share this trunk (Fig. 2a): a *pose head* ($256 \rightarrow 128 \rightarrow P$ logits), a *form head* ($256 \rightarrow 64 \rightarrow 1$, a correctness probability), and a *deviation head* ($256 \rightarrow 128 \rightarrow 15$ per-joint deviations). Sharing the trunk makes one forward pass of about 0.36 M parameters answer “which pose, how well, which joints” in a few milliseconds on a CPU.

5.2 The cascade: one model names, another vetoes

Through the original decision path, a rule engine overrode the MLP on every disagreement, so replacing the MLP with a better one changed nothing (identical to rules-only, Table 5). We therefore split the roles. The *namer* is the earlier photo-trained 3-head MLP (v4), whose top pose is the answer. The *gate* is a newer 3-head MLP of the same architecture trained on the cue-verified video table plus training photos and photos of other poses (labelled transition/unknown); it answers “is this one of my poses at all?” and supplies the form score. The rule is fixed in advance: the answer is v4’s top pose unless the gate’s top class is transition/unknown, in which case the system says it does not recognise a pose. This is a reject option in the sense of selective classification [21], applied at the system level. The policy was chosen on a validation half of the held-out photos and confirmed on the untouched half (Section 8).

5.3 Rule engine and per-joint cues

Each pose has a table of angle bands (e.g. for Downward Dog: knee $\in [110^\circ, 180^\circ]$, shoulder $\in [95^\circ, 180^\circ]$, hip $\in [20^\circ, 140^\circ]$). The deviation of a joint is the distance in degrees outside its band (0 if inside), and the rule score is $1 - \bar{d}/45^\circ$ clipped to $[0, 1]$. Because the learned deviation head turned out to be weak (Section 8), the per-joint cue and the colour of each limb in the skeleton overlay come from these bands wherever a pose has them (17 of the 19 poses): coral if a joint is more than 15° outside, amber above 7° , sage otherwise (Fig. 3a). Two poses, Tree and Lunge, have no bands. Until 2026-10-07 the cascade fell back to the learned head for them, so a correct Tree could show a coral leg (Fig. 3d); we found this while verifying the calibration feature, and the deployed system now reports no per-joint deviations for these two poses and draws neutral limbs (Section 11).

5.4 ST-GCN “flow check”

The sequence model treats a 60-frame window as a graph signal on the 33-joint MediaPipe skeleton. Coordinates are centred on the pelvis and divided by the hip width. Each of three ST-GCN blocks [7] applies a graph convolution with the symmetric-normalised skeleton adjacency $\mathbf{A}_{\text{norm}} = \mathbf{D}^{-1/2}(\mathbf{A} + \mathbf{I})\mathbf{D}^{-1/2}$ (a 1×1 projection followed by aggregation over neighbours) and a 9-tap temporal convolution, with GELU and a residual connection; channels grow $3 \rightarrow 64 \rightarrow 128 \rightarrow 256$, features are averaged over joints and time, and a two-layer head outputs eight target poses plus transition/unknown (Fig. 2b). The app shows a flow-check answer only if its confidence reaches 0.70 (0.55 for child’s pose); otherwise the UI shows “static check”, and the per-frame cascade alone names the pose and scores form.

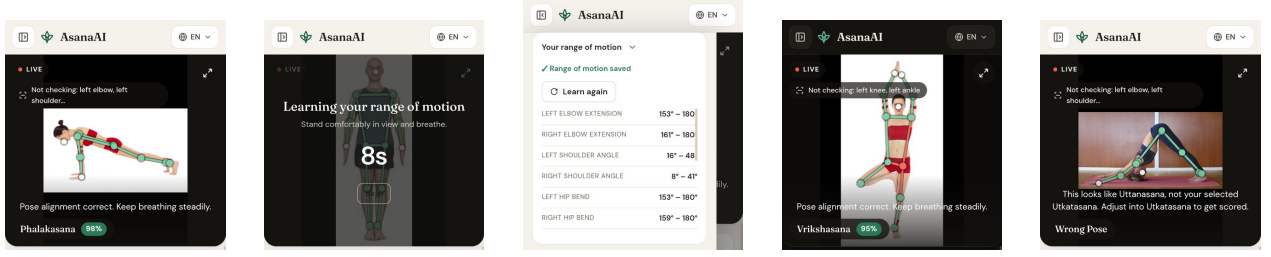
The input-rate contract. The model is trained on 60 consecutive native-rate frames (≈ 2.4 s at 25 fps). An early version of the web app fed its buffer one frame per finished server round trip (at most 2 fps), so its “60 frames” spanned 30–120 s and the model looked broken. The app now fills a time-based buffer from every camera frame and resamples it to 25 fps; Section 8 quantifies why this matters.

6 Occlusion, Personalisation and Safe Coaching

6.1 Visibility-aware scoring

An audit of the first version showed that the server’s mesh-recovery branch was never wired (nothing sent it data and no such model was deployed), and what actually ran for hidden joints was a left–right mirror: exact for symmetric stances, wrong for asymmetric ones. In Tree pose the raised foot was placed on the floor, off by 0.23 of the frame height, and still reported as “recovered”. Meanwhile the joint angles were always computed from the raw landmarks, so a hidden joint could still produce a confident wrong score and cue.

We replaced guessing with abstaining. Joints whose landmark visibility is low (smoothed with hysteresis in the browser) are neither scored nor coached; the interface lists what it is not checking (Fig. 3a) and draws hidden limbs at 25% opacity. The map from hidden joint to affected angle features was checked against the Python angle code by perturbation (0 visibility errors). A known limit remains: the pose name can still be influenced by a hallucinated hidden joint; only scoring and coaching are masked.



(a) Hidden joints are not judged

(b) Calibration: 15 s count-down

(c) Saved range of motion

(d) Failure: false coral leg

(e) Failure: wrong name

Figure 3: Live screenshots of the deployed app on a POCO M7 Pro over USB, with a credited Wikimedia photo standing in for the camera (no live person). (a) Plank, guided mode: visible limbs are sage (inside their angle bands) and the chip “Not checking: left elbow, left shoulder” names joints the camera cannot see, which are drawn faint and neither scored nor coached. (b) Calibration asks the user to stand comfortably for 15 s. (c) The learned per-joint ranges are saved on the device (the still photo yields narrow ranges). Two failures shown on purpose: (d) a correct Tree (guided, dark theme) with a coral standing leg, captured before the fix: Tree has no angle bands, so its joint colours then came from the learned deviation head, which is unreliable; the deployed app now draws neutral limbs for Tree and Lunge (Section 5); (e) a Downward Dog photo named Uttanasana (a forward fold), after which guided mode answers “Wrong Pose”. Photos: Kennguru (a, d), Witold Fitz-Simon (b, c; CC BY-SA 2.5), Iveto (e); CC BY 3.0 unless noted, via Wikimedia Commons.

Table 2: Recovering hidden joints (141 cases; lower is better). Errors are relative to MediaPipe on the clean frame; NME is 2-D distance over torso length. Bold: better of the compared methods within each subset.

Subset (n)	Method	Med. angle	Mean angle	Med. NME
All (141)	raw	17.9°	38.9°	0.291
	mirror	8.6°	20.2°	0.152
	CLIFF	5.8°	16.8°	0.185
	CLIFF aligned	7.0°	22.0°	0.179
Symmetric (117)	mirror	5.2°	13.8°	0.131
	CLIFF	6.1°	11.0°	0.173
Tree, asym. (24)	mirror	20.1°	51.2°	0.470
	CLIFF	4.8°	44.8°	0.357
Left leg (47)	mirror	16.2°	33.5°	0.214
	CLIFF	6.2°	12.7°	0.270
Right arm (47)	mirror	4.9°	11.9°	0.092
	CLIFF	15.1°	30.7°	0.210
Both feet (47)	mirror (=raw)	6.1°	15.1°	0.199
	CLIFF	4.7°	6.9°	0.143

6.2 Would mesh recovery do better? CLIFF versus the mirror

Mirroring is crude, so we asked whether CLIFF [8] (a ResNet-50 model that predicts an SMPL body [20] from the full frame) recovers hidden joints better. On 70 real yoga frames in four poses, with MediaPipe on the clean frame as the reference, a grey box hid one limb (left leg, right arm or both feet; 141 cases) and the hidden joints were recovered four ways. Table 2 gives the errors. Neither method dominates. The mirror is better where the body really is symmetric (arms, mountain, seated) and has the lowest median landmark error overall; CLIFF is better for legs, for both feet hidden and for the asymmetric Tree pose, and its median joint-angle error is 5.8° versus 8.6°. But CLIFF costs 164 ms per image on two CPU threads (18.7 ms on a T4 GPU) and, decisively, needs the image, which the app never sends. We therefore do not ship it and keep the abstain policy. Caveats: 141 cases from 70 frames, one asymmetric pose, a synthetic occluder, a MediaPipe reference rather than motion capture, and a neutral body-model stand-in for an unavailable initialisation file.

6.3 Personal range-of-motion calibration

Bodies differ: a joint inside its owner’s comfortable range is “different”, not “wrong”. At camera start the app offers a 15-second calibration (Fig. 3b): the user stands comfortably in view while the 15 joint angles are recorded. For each joint the profile stores the observed minimum and maximum widened by 15° (clamped to $[0^\circ, 180^\circ]$) and the mean as the resting angle; if fewer than 20 frames were captured it is discarded rather than “pretend we calibrated”. The profile lives only in the browser’s local storage (Fig. 3c); each request carries the numbers. On the server, a joint whose angle lies inside its personal range has its deviation set

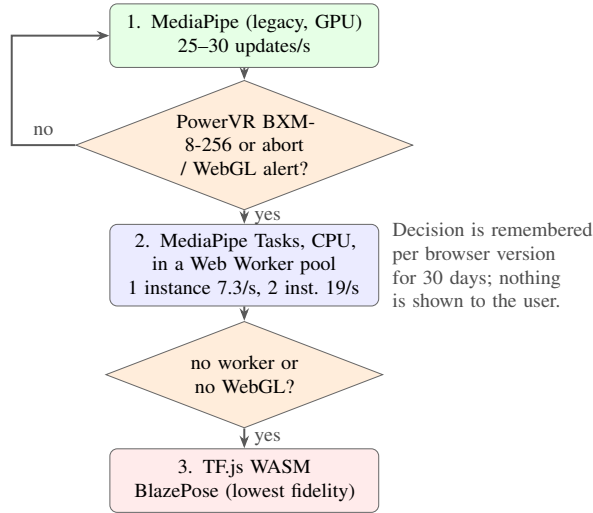


Figure 4: The pose-engine fallback ladder. The choice is automatic, remembered per browser version for up to 30 days, and invisible to the user.

to zero, and a personal form score is computed that closes the fraction of the gap to a perfect score that the forgiven deviation represents; it is therefore never lower than the universal score, and equal when nothing is forgiven. The UI shows it as “For your body” beside the universal score. We verified the mechanism against the deployed server with a hand-built skeleton (angles from the app’s own function; both hip deviations 56° from their bands, universal score 0.924). Without a profile the response carries no personal score; a profile not containing the current angles leaves it at 0.924; one containing the left hip’s angle zeroes that deviation (0.962); both hips give 1.000; the universal score never changed. A profile spanning 0° – 180° for every joint also gives 1.000, which shows how much a poor profile can forgive. The feature has not been validated with users or clinicians (Section 11).

6.4 Safe, multilingual coaching

The cue sentence goes through three stages. (1) A symbolic stage picks the joint furthest outside its band and a reviewed template for it in English, Hindi or Bengali. (2) Optionally, a hosted language model (a 27-billion-parameter Qwen model, temperature 0.2) paraphrases the template and may only rewrite it. (3) A post-generation screen discards the paraphrase and keeps the template if it contains any of “push”, “force”, “hurt”, “pain”, “stretch more” or “further”. A cue that is delivered and produces no measured improvement escalates: the second time it adds how many degrees off the user is, and from the third time it backs off (“ease out of the shape and reset; come back only as far as feels comfortable today”), because a cue that failed twice usually means the range is not available today. A lesson from operating this stage: when the provider retired the model name we had configured, every request silently fell back to the template and the failure looked identical to success until the response was found to be byte-identical to the template. The fallback is safe, which is exactly why a silent one is dangerous; the code now logs every non-200 answer.

7 Deployment on Real Phones

The failure. On a POCO M7 Pro (MediaTek Dimensity 7025, PowerVR BXM-8-256 GPU, Android 16, Chrome 154) the web app showed a raw “Failed to create WebGL canvas context” alert. This is not “no WebGL”: Chrome creates WebGL2, WebGL1 and OffscreenCanvas contexts fine. The legacy MediaPipe pose engine then aborts inside the GPU driver (a shader-uniform assertion in its image scaler) and every later frame throws; MediaPipe Tasks with the GPU delegate runs in 16 ms per frame but returns no poses. Driver problems of this GPU family are discussed on the vendor’s developer forum,¹ and the alert itself is a alert box inside MediaPipe’s script.² Because fixes must be automatic, we measured candidate engines over USB with the Chrome DevTools Protocol, touching only a tab we create.

¹<https://forums.imgtec.com/t/bxm-8-256-long-list-of-driver-issues/3891>

²<https://github.com/google/mediapipe/issues/2381>

Table 3: Pose engines measured on the POCO M7 Pro (lite models unless stated).

Engine / configuration	Measured
Legacy MediaPipe, GPU (PowerVR BXM-8-256)	aborts in driver
MediaPipe Tasks, GPU delegate	16 ms, no poses
Tasks, CPU, image mode	~176 ms/frame
Tasks, CPU, video mode (lite / full)	~80 / ~117 ms
... in a Web Worker, round trip	85–95 ms
Worker pool, 1 / 2 / 3 instances	7.3 / 19 / 24 results/s
TF.js WASM BlazePose (last resort)	~95 ms; 4.2–6.7° mean angle gap

Table 4: Pose-head generations on clean tests (bold: best in column). Frozen Commons-103: none of these models trained on those photos (the new model is scored out-of-fold). Held-out public photos: 1,685 (1,037 other poses); v4 may have seen similar public images, which can only flatter it.

Model	Frozen Commons-103		Held-out public	
	Overall	Macro rec.	Poses pass	False alarm
19 Jul original	23.3%	17.5%	0	75.0%
3 Sep retrain	13.6%	11.1%	0	64.5%
21 Sep v4 (was live)	60.2%	64.8%	1	81.5%
New (gate model)	49.5%	58.0%	3	22.6%

The ladder. The app tries MediaPipe on the GPU; if the renderer is the known-bad chip (recognised at page load) or the engine aborts or raises the alert, it switches to MediaPipe Tasks on the CPU inside a Web Worker (a second MediaPipe module cannot share the page with the legacy one because of a global-namespace collision, hence the worker); if no worker or WebGL is available it falls back to TensorFlow.js [19] with a WebAssembly BlazePose (Fig. 4). Table 3 gives what we measured. A single worker gave 7.3 results/s; phones with at least six cores now run two instances and alternate frames (late answers for older frames are dropped), which raised the skeleton from about 8–10 to about 22 updates/s in the running app. The last-resort engine is lower in fidelity: against the CPU engine its mean joint-angle difference was 4.2–6.7° (worst 27°) and it missed a plank. On a desktop, the GPU and CPU engines differed by 2.4° (Tree), 2.5° (Cobra) and 4.8° (Warrior II) on the same photos. The same fallback also worked in a second browser (Brave) that masks the GPU name, where recognition up front is impossible and the reactive path took over after the driver abort, then drew the skeleton and recognised Warrior II at 98% (one photo).

Offline and degraded operation. When the server is unreachable, the browser falls back to *basic mode*: a TypeScript port of the rule engine and coaching templates. We proved the port against the Python original with generated cases: 10,000 frame cases and 2,500 coaching-text cases over 21 poses, 0 mismatches. Basic mode is rules only—no MLP, no gate, no ST-GCN, no language model—so its pose naming equals the “rules only” row of Table 5, not the cascade; the interface says so and says why (offline, server waking, reconnecting). The first answer of a session is local so the user never waits for a sleeping server, API calls time out after 9 s, and pose and score refresh about every 1.5 s (every 10 s in the first version).

8 Results

All numbers below were measured on data the models never trained on (Section 4); where a caveat applies it is stated next to the number.

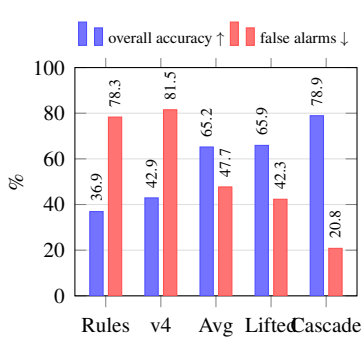
8.1 Naming the pose from one frame

Progress over model generations. Table 4 follows the pose model from the first (about two-month-old) model to the current one. On the held-out public photos the earliest models passed no pose; the current gate model passes three with 22.6% false alarms. The 86–90% that two September models score on all 422 Commons photos is deliberately omitted because they were trained on most of those photos.

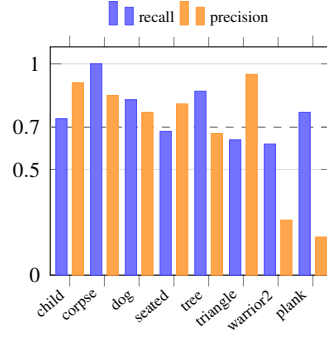
The decision path matters more than the model. Table 5 and Fig. 5a compare decision policies on the 1,685 held-out public photos using the real production functions. Through the original path (rule engine wins every disagreement) a new MLP changed nothing: identical to rules-only (36.9%, 78.3% false alarms, no pose

Table 5: Decision policies on the 1,685 held-out public photos (bold: best in column). False alarm: a photo of another pose named as one of ours (lower is better).

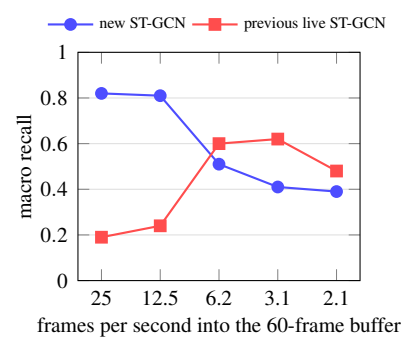
Policy	Overall	False alarm	Poses pass
Production before (v4 + rules)	36.9%	78.3%	0
New model + rules	36.9%	78.3%	0
Rules only	36.9%	78.3%	0
v4 alone	42.9%	81.5%	1
Average of both MLPs	65.2%	47.7%	5
Veto lifted when rules agree	65.9%	42.3%	5
Cascade (v4 names, new model vetoes)	78.9%	20.8%	7



(a) Decision policies (held-out public photos)



(b) ST-GCN, held-out videos, per pose



(c) ST-GCN input-rate contract

Figure 5: (a) Rules-only, v4, the average of both MLPs, the veto lifted when rules agree, and the cascade. (b) Per-pose recall and precision of the final ST-GCN on videos it never trained on (dashed: 0.70 bar; warrior 2 and plank have few held-out windows). (c) Macro recall on held-out videos when the 60-frame buffer is filled at different rates; the new model needs training-rate windows, the previous model was partly trained on slow windows.

passing). The cascade reaches 78.9% with 20.8% false alarms and seven poses at recall and precision ≥ 0.70 (chair, corpse, downward dog, tree, triangle, upward dog, Warrior II). The policy was chosen on a validation half only (79.3%, 20.1%, 3 poses) and confirmed on the untouched half (78.6%, 21.6%, 4 poses: corpse, downward dog, tree, Warrior II). On the wild frozen-103 set the cascade scores 55.3% against 37.9–40.8% before; v4 alone is better at pure naming there (60.2%), so the cascade gives up about five photos of naming accuracy for about 60 points fewer false alarms elsewhere. Through the real frame-analysis endpoint with the real models, switching the cascade off reproduced production exactly and switching it on reproduced 78.9%/20.8%/7, agreeing with the offline policy on 100% of requests; a replay against the live Space gave 78.1%/21.8%/7 (16 of 1,685 requests were rejected by the host’s rate limiter under an 8-thread stress test).

8.2 Baselines: is the neural network necessary?

To answer “compared with what?” we trained standard methods on exactly the rows the new MLP was trained on (136,506 rows of 15 zero-z angles: cue-verified video frames, training photos and photos of other poses labelled transition/unknown) and scored them on the same photos with the same metrics: k -NN ($k=15$, distance-weighted), an RBF SVM, logistic regression, a 300-tree random forest, and a plain MLP (256–256–128, ReLU, no residual blocks or batch normalisation). The SVM, forest and logistic regression use class-balanced weights; the SVM sees at most 20,000 class-balanced rows, the other methods between 64,000 and 115,000. These runs use one random seed and were executed on Kaggle. Table 6 and Fig. 6 report two held-out sets: the 1,685 public photos (same public datasets as the training photos, disjoint files) and the frozen *wild* Commons-103 photos from another source, scored out-of-fold. *Findings.* (1) Linear, nearest-neighbour and kernel models are poor at telling “another pose” from “one of mine” (false alarms 56–96%). (2) A plain MLP matches the residual 3-head model on the public photos (80.9% against 79.6%): the residual trunk is not what buys accuracy, and what the 3-head design adds is the form and deviation outputs from one small forward pass. (3) A random forest is better than the cascade on the public held-out photos: 85.3% against 78.9% overall and 9.8% against 20.8% false alarms, with seven passing poses each (nine in an earlier run with a different random subsample; the number of poses clearing a fixed bar flips with small changes, so we read little into it).

Table 6: Standard methods on the same data and held-out photos (bold: best in column). Public: 1,685 held-out photos (1,037 other poses). Wild: frozen Commons-103, out-of-fold (a different source; with 103 photos, gaps under ~ 10 points are within noise). [†] first of two runs.

Method	Public held-out			Wild
	Overall	False alarm	Poses pass	Overall
Production before (v4 + rules)	36.9%	78.3%	0	37.9%
Logistic regression [†]	29.7%	95.8%	1	–
k -NN	58.6%	55.7%	1	50.5%
SVM (RBF)	58.4%	56.2%	3	52.4%
Plain MLP	80.9%	19.1%	3	47.6%
Random forest	85.3%	9.8%	7	35.0%
v4 alone	42.9%	81.5%	1	60.2%
New 3-head model alone	79.6%	22.6%	3	49.5%
Cascade (ours)	78.9%	20.8%	7	55.3%

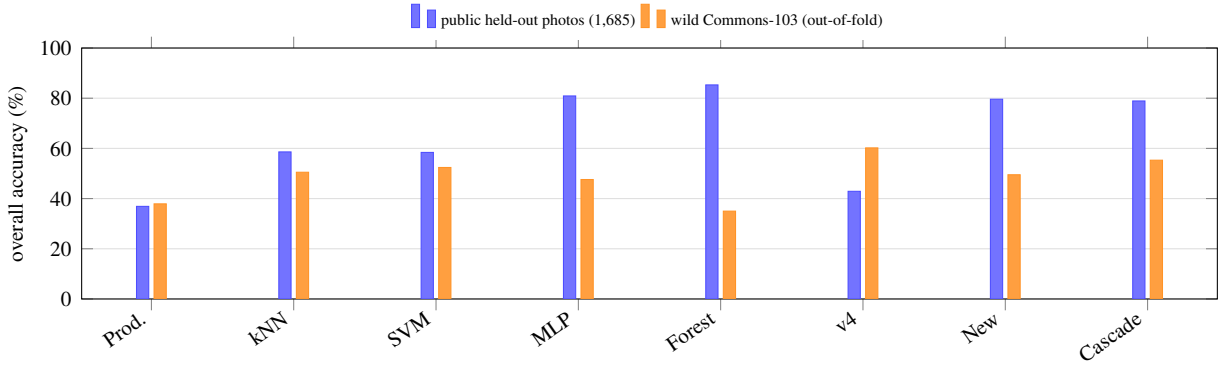


Figure 6: The ranking of methods changes with the photo source. Prod.: production before (rules); MLP: plain MLP; New: the new 3-head model alone; Cascade: ours. The forest leads on photos from the training sources and trails every learned method on wild photos.

Is the forest’s lead a memorisation artefact? We audited the public test set by the distance of each photo to its nearest training photo in standardised angle space. Exact near-duplicates are rare: only 0.3% of the 1,685 photos lie within 0.05 of a training photo and 4.4% within 0.2. The forest’s lead also holds in every distance quartile, including the farthest (Q4: 84.6% against 77.0% for the cascade, Table 7), although every method is most accurate on photos closest to training photos. But the public held-out photos come from the same public datasets as the training photos, and on the wild Commons-103 set the ranking reverses: the forest is the weakest learned method (35.0%), 13–25 points below the SVM (52.4%), k -NN (50.5%), plain MLP (47.6%), the new network (49.5%), the cascade (55.3%) and v4 (60.2%). We read this as: tree ensembles are the strongest method on photos like the training photos and the weakest on photos from another source, so no method is best on both. The cascade is within seven points of the best method on each set and keeps false alarms at 20.8%; a tree–network ensemble is the obvious next experiment (Section 11). We keep the 3-head network because the form and deviation heads, the 0.36 M-parameter footprint and the better behaviour on wild photos matter for a coach that must run on any phone, and because it is the architecture of the project proposal; we do not claim it is the best frame-level namer on same-source photos.

8.3 Held-out videos: the ST-GCN

On three folds split by video (24 videos, 8,002 windows, 60 frames, stride 12), the final target-pose ST-GCN reaches 82.2% overall; because 60% of the windows are transition/unknown this flatters it, so we also report 78.0% macro recall over the nine classes (76.9% over the eight poses). Three poses clear the bar: child’s pose (recall/precision .74/.91), corpse (1.00/.85) and downward dog (.83/.77); seated easy, tree and triangle are within about 0.07 of it (Fig. 5b). The model is conservative: its misses go to transition/unknown instead of to another pose. Warrior II and plank fail on precision (.26, .18): 158 transition windows are called Warrior II and plank has only 30 windows. A less conservative threshold chosen without leakage (an offset on the transition logit, picked on two folds and applied to the third) leaves overall accuracy at 81.9% and raises the passing poses to four (child, downward dog, triangle, seated easy; corpse then drops below the precision bar);

Table 7: Public held-out photos split into quartiles of distance to the nearest training photo (Q1 closest; 421–422 photos each): overall accuracy / false-alarm rate.

Method	Q1	Q2	Q3	Q4
Random forest	93.1 / 16.2	82.7 / 19.6	81.0 / 9.2	84.6 / 1.7
Plain MLP	91.7 / 14.8	78.6 / 28.0	75.8 / 21.3	77.7 / 13.2
New 3-head model alone	90.5 / 16.2	77.9 / 32.9	74.1 / 25.2	75.8 / 16.3
Cascade (ours)	89.1 / 16.2	76.2 / 32.0	73.4 / 23.2	77.0 / 13.5

Table 8: ST-GCN versions on the same held-out windows (12 videos new to the old models; bold: best in column). “Answered” is the share of true-pose windows that reach the UI, with its precision.

Model	Overall	Macro rec.	Passing	Answered / prec.
Old original (Jul)	67.3%	11.6%	0	0% / –
Old v2	67.8%	13.2%	0	1.8% / 100%
Old live	68.9%	18.8%	0	2.8% / 93.8%
New	76.9%	82.1%	2	50.5% / 88.1%

no offset reaches six. We therefore claim three poses (four with the cross-fitted threshold) for the ST-GCN and keep the “seven poses” claim for the frame cascade.

Old versus new. Table 8 scores the sequence model that was live until this work against the new one on the same held-out windows. The 12 videos that arrived after the old models were trained make that subset clean for them. The old live model had macro recall of 18.8% and answered (with confidence $\geq$ its gate) on 2.8% of true-pose windows; the new model answers on 50.5% at 88.1% precision. The old model’s named transitions are not measured here and the new model has a single generic transition class.

The input-rate contract. Fig. 5c re-scores the same videos while subsampling every k -th frame. The new model keeps macro recall 0.82 at the training rate (25 fps) and 0.81 at 12.5 fps, then loses it (0.51 at 6.2 fps, 0.41 at 3.1 fps, 0.39 at 2.1 fps); the old model, partly trained on slow windows, is better at slow rates but never exceeds 0.62. This is why the web app now fills a time-based 25 fps buffer. A server audit found the backend architecture and normalisation identical to the trainer’s (maximum logit difference 0.0 on 64 windows) and the deployed endpoint matching a local run of the same weights on 40/40 and 60/60 windows, so the weakness of the old model was generalisation, not serving. *Sequence baselines (GPU run, Kaggle T4, 1915 s).* Six sequence models were trained under one recipe (AdamW, cosine schedule, class-weighted loss, 10 epochs, at most 6,000 windows per class, last epoch scored, one seed) and scored on the same three video folds. Results (overall / macro recall over nine classes / over eight poses / poses passing): LSTM 83.0% / 89.5% / 91.0% / 4; BiLSTM with attention 87.1% / 89.6% / 90.1% / 5; temporal CNN 81.5% / 89.7% / 91.4% / 4; MLP on window mean and standard deviation (86 k parameters) 85.3% / 94.4% / 96.4% / 5; ST-GCN without the skeleton graph 77.1% / 84.8% / 86.4% / 4; ST-GCN 76.0% / 86.4% / 88.6% / 4. The skeleton graph helps only slightly over the graph-free variant, and simpler models match or exceed the ST-GCN; an ST-GCN fold trains in about 290 s against 6–15 s for the others. Raw results: [planning/kaggle_transfer/kernel_baselines_seq_gpu_arko/results_seq_gpu_v1.json](https://planning.kaggle-transfer/kernel_baselines_seq_gpu_arko/results_seq_gpu_v1.json).

8.4 Does the form score mean anything?

No labelled good-versus-bad-form data exist for these poses, so we use a property test: on 648 held-out in-vocabulary photos, five trials each, one joint is broken by 45° . Table 9 shows that the new gate’s form score is high on good form (0.81) and drops in 85.0% of trials, the best of the four models, while the deviation head’s top-ranked joint is the broken one in only 12.4% of trials (chance 6.7%), which is why per-joint cues come from the angle bands. The same test on the deployed endpoint with real MediaPipe angles from eight instructor clips gives a mean score of 0.94 while held, 0.25 with the left knee broken (a drop on every probe of every clip) but 0.92 with the left elbow broken: the score collapses for leg errors and is nearly blind to arm errors. Arm errors can only be caught, if at all, by the angle-band layer, and we do not claim whole-body form checking. (These clips are in the gate’s training data, so this is a regression check of the deployed path, not a generalisation number.)

Table 9: Property test: one joint broken by 45° (chance for “points at the broken joint” is 6.7%; bold: best in column).

Model	Score, good form	Score drops	Top joint correct
19 Jul original	0.28	65.0%	8.1%
3 Sep retrain	0.67	74.9%	18.2%
21 Sep v4	0.79	59.2%	6.7%
New	0.81	85.0%	12.4%

Table 10: On-phone photo test (photos correct / tested; a pose passes at 50% or more). Two photos had no person found.

Pose	Correct	Pose	Correct
Tree	6/6	Chair	3/5
Triangle	5/5	Corpse	2/4
Warrior II	3/3	Child’s pose	1/3
Downward dog	4/4	Cobra	1/5
Seated easy	4/5	Mountain	0/5
Plank	3/4		

8.5 End-to-end checks on a real phone and on recorded video

On-phone photo test. With the production app running on the POCO M7 Pro through its CPU engine and a Wikimedia photo standing in for the camera, every answer of the live server was logged (Table 10); a photo counts if the most frequent of three answers is the expected pose, and a pose passes at $\geq 50\%$ of its photos. Eight of eleven poses pass; of the seven poses that passed the held-out benchmark, the six tested here all pass, while mountain, cobra and child’s pose do not (mountain and cobra were already weak). These are still photos with labels taken from file titles, and the models trained on Commons photos, so some may have been seen: the test checks the on-phone path (engine, landmarks, server, label), not generalisation.

Live replay on recorded video. We replayed eight clips of held poses (six poses, real MediaPipe on instructor video) against the deployed Space as the app would call it: seven of eight clips scored ≥ 0.81 frame-level accuracy while the pose was held (call-weighted 88.4%), with zero failed API calls. One Seated-easy clip failed (0.06: named upward dog throughout), so Seated easy is not reliable live. On three clips of continuous movement the system reported “transitioning” for 74%, 92% and 79% of frames. All eight clips come from videos in the training data of the gate, so this is a plumbing check; the ST-GCN, deployed after, answered correctly on 100% of calls for the four poses it knows and stayed “transition/unknown” for mountain and lunge, which are outside its vocabulary.

8.6 The interface

Fig. 7 shows the deployed interface on the phone in the three languages and both themes, in free and guided mode, and the public landing page. The skeleton overlay uses the colours of Section 5 and the score ring, “For your body” personal score and coaching cue update about every 1.5 s.

9 Previous Framework Versus the Current Framework

The project started from a two-model design (a 3-head ResMLP and an ST-GCN) trained on a rule-derived table, with the rule engine deciding every disagreement. This section sets the previous framework against the current one, component by component. Every number is measured on photos or videos the model never trained on, with the same test set on both sides; the sources are Sections 8 and 10 and docs/BENCHMARKS.md. Where the current framework is not better, that is stated in the last paragraph.

What produced the gain. Three changes account for almost all of it. (i) The labels: the previous table named a pose wherever the rules said so, so a model could at best copy the rules, and the rules won every disagreement; the new labels need the instructor’s spoken pose name and the rules and a steady hold, and the features are computed the way the app serves them. (ii) The decision path: the previous path let the rule engine overrule the model, so swapping in a better model changed nothing (identical 36.9% and 78.3%); the cascade lets a photo-trained model name the pose and a second model veto it when the pose is not one of ours, which is what removed most false alarms. (iii) The ST-GCN data and input rate: the previous live model scored

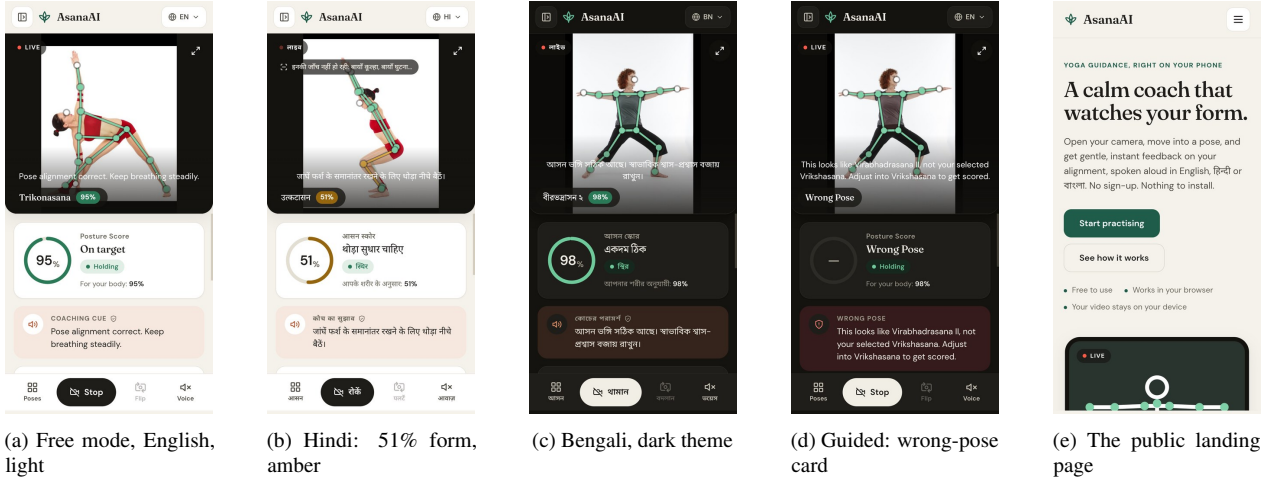


Figure 7: The deployed interface on a POCO M7 Pro, a credited Wikimedia photo standing in for the camera (no live person): (a) Trikonasana, 95%; (b) Utkatasana with the Hindi cue, 51% in amber and a chip naming the joints not checked; (c) Virabhadrasana II in Bengali; (d) guided mode with Vrikshasana selected while a Warrior II photo is shown: the score is withheld and the app names the pose it sees; (e) the public landing page (light theme). Photos by Kennguru (a, b; CC BY 3.0) and lululemon atletica (c, d; CC BY 2.0), via Wikimedia Commons.

18.8% macro recall on held-out videos and was confident on under 3% of windows; the browser also fed it frames at up to 2 per second instead of 25, which is a different input from what it was trained on. The current model is trained on cue-verified windows with mirror and photo-hold augmentation, and the web app now resamples every camera frame to 25 per second. At the old feed rate of about 2 frames per second neither model is usable (macro recall 0.48 for the previous model and 0.39 for the current one), so the buffer fix is what makes the temporal model usable at all.

Where the current framework is not better. (a) The per-joint deviation head is still weak: the broken joint is its first choice in 12.4% of trials against 6.7% by chance, so joint-level cues come from each pose’s angle bands, not from that head. (b) On wild Commons photos the previous MLP used alone names 60.2% against 55.3% for the cascade; the cascade gives up about five photos of naming accuracy for roughly 60 points fewer false alarms. (c) The previous ST-GCN’s few confident answers were more precise (93.8% against 88.1%), but they covered 2.8% of the windows against 50.5%. (d) The previous ST-GCN was trained to name transitions: its encoder holds 30 classes (21 holds, seven directional transitions such as `transition:lunge_pose->warrior_2`, `transition:other` and unrecognized). The shipped model has one generic transition class, because the cue-verified labels cannot name transitions (Section 9.3). The code comments and training notes quote 24 or 25 classes for this model and a macro recall of 63.0% on two held-out videos with rule-defined labels; neither the class count nor that figure matches the checkpoint now deployed, so we do not use them. (e) A random forest trained on the same photos beats the whole cascade on same-source photos (85.3% against 78.9%) and loses on wild photos and on false alarms (Section 8); we judge the cascade by the wild and false-alarm columns because those are the conditions of a phone camera.

9.1 The MLP, generation by generation

The pose-naming MLP has been retrained several times, and every generation is scored on the same held-out data: Table 4 (naming and false alarms, from the two-month-old original to the current model), Table 5 (the decision path around it) and Table 9 (does the form score react to a broken joint). In short: the original passed no pose and named a pose for 75.0% of photos of poses we do not know; the first retrain was worse on wild photos (13.6%) before the photo-domain data arrived; the 21 September model names wild photos best (60.2%) but names a pose for 81.5% of the unknown ones; the new gate model is the opposite (49.5% on wild photos, 22.6% false alarms, three poses passing); and the cascade built from the two passes seven poses with 20.8% false alarms. No single MLP is best everywhere, which is why the current framework is a pair of small MLPs and not a replacement of one by another. The correctness score comes from the new gate model, which scores 0.81 on correct form and drops by 85.0% when one joint is broken, against 0.79 and 59.2% for the 21 September model.

Table 11: Previous versus current framework (held-out data only; bold marks the better value of each row).

Measure	Previous	Current	Test set
Training labels	654,488 rule-derived rows; 3-D angles in training, 2-D angles served (median mismatch 14°)	cue-verified: instructor names the pose, rules agree, pose is held; served 2-D angles	24 videos, 1,206,391 frames
Overall naming accuracy, decision path in production	36.9%	78.9%	1,685 held-out public photos
False alarm (a pose outside the vocabulary named as one of ours), lower is better	78.3%	20.8%	same 1,037 other-pose photos
Poses passing the 0.70 recall and precision bar	0	7	same photos
Naming on wild photos	37.9%	55.3%	frozen Commons-103, out of fold
Score drops when one joint is broken by 45°	59.2%	85.0%	648 held-out photos $\times$ 5 trials
Score on correct form (higher is better)	0.79	0.81	same trials
ST-GCN macro recall on a rule-defined movement task (19 classes)	14.2%	53.1%	9,458 windows of the same 12 videos; the order-blind MLP scores 66.7%
ST-GCN macro recall on held poses (the task the app’s sequence row shows)	18.8%	82.1%	2,682 windows of 12 videos the old model never saw; outputs mapped to the 8-pose vocabulary
Share of true-pose windows the ST-GCN answers confidently (precision of those answers)	2.8% (93.8%, 32 answers)	50.5% (88.1%, 611 answers)	same windows
ST-GCN macro recall at the training rate (25 frames/s)	0.19	0.82	same windows
Hidden joints	mirrored from the other side (median angle error 8.6°)	visibility-aware: ab-stains rather than guesses; 5.8° for the offline CLIFF experiment, not shipped	occlusion test, Section 6
Poses with joint-level cues	17 of 19, wrong joints marked red on Tree and Lunge	17 of 19, no red on the two without bands	live check on the deployed app

9.2 Is the sequence model a genuine ST-GCN?

We checked this directly because a sequence model that is not a graph network but is called one would invalidate every sequence result. The evidence, from the repository and the training logs:

- **The code.** The model in `backend/app/models/sequence.py` applies, in each of three blocks, a 1×1 convolution followed by multiplication with the symmetric-normalised 33×33 adjacency of the MediaPipe skeleton (`einsum('vw,ncwt->ncvt')`), then a 9-tap temporal convolution with batch normalisation and a residual path.
- **The parameter count.** Counting the layers by hand (blocks $3 \rightarrow 64 \rightarrow 128 \rightarrow 256$, head $256 \rightarrow 128 \rightarrow 9$) gives 893,897 parameters, which equals the count printed by the benchmark run for the 9-class model. A recurrent model of this size has a different count and different layer names.
- **Strict loading.** The deployed loader and the old-versus-new benchmark load every checkpoint into this class with `load_state_dict`, which fails on any checkpoint with different layer names; all three previous checkpoints loaded and were scored.
- **An ablation that changes the graph.** Replacing the adjacency with the identity matrix (no skeleton graph) changes macro recall from 84.8% to 86.4% over nine classes, so the graph is used by the network, though the gain is small (one seed).
- **History.** The repository’s first commit (10 June 2026) held a different model under the same class name: a 1-D convolution, two residual GRU blocks and attention, with no skeleton graph. It was replaced by the graph network on 17 July 2026 (commit 018a7ba), and the class name `YogaSequenceLSTM` was kept

Table 12: Effect of the depth coordinate: macro recall of the same models trained with the full (x, y, z) skeleton and with z set to zero (bold: best in column). Held poses: nine classes on the cueT2 video folds; Movement: the 19-class rule-defined task. At most 3,000 training windows per class and one seed, so the full-skeleton held-pose values differ slightly from the sequence-baseline table of Section 8.

Model	Held poses		Movement	
	x, y, z	$z=0$	x, y, z	$z=0$
MLP on window mean/std	94.2%	59.3%	59.5%	46.3%
Temporal CNN	90.5%	70.1%	56.2%	52.1%
ST-GCN	88.7%	28.6%	47.5%	35.9%

so existing checkpoints would load. Results for any checkpoint produced before the replacement would be results for that GRU model; this is why the old-versus-new comparison loads the old checkpoints with strict loading into the graph class.

Why this architecture. The design notes record the split of work: the MLP checks one frame (pose, correctness, per-joint deviation) and the ST-GCN reads 60 frames for transitions and flow, because a single-frame classifier can judge alignment but not movement [7]. A graph network suits that job for three reasons: it models time (temporal convolution), the structure of the skeleton (graph convolution) and 3D, since it is the only model fed the full (x, y, z) landmark coordinates while the MLP receives angles computed with z set to zero because the camera’s depth is the least reliable coordinate (Section 4.2). The project proposal fixed this pair of models; the proposal itself is not in the repository, so these reasons are taken from the design notes and from the team. What we measured for each reason: *time*: on held poses a window-statistics MLP that ignores frame order matches the ST-GCN, and movement cannot be tested with our labels (Section 9.3); *graph*: removing the skeleton graph changes nothing measurable; *3D*: the depth coordinate is the largest effect we measured (Table 12). Setting z to zero lowers every sequence model on both tasks: on held poses the MLP on window statistics falls from 94.2% to 59.3% macro recall, the temporal CNN from 90.5% to 70.1% and the ST-GCN from 88.7% to 28.6%, where it collapses onto the generic transition class; on the movement task the losses are 13.2, 4.1 and 11.6 points. An older check on two held-out videos with rule-derived labels agrees in direction (ST-GCN macro recall 54.6% with raw z , 49.7% with bone-length-corrected z , 44.8% with z zeroed). So the project did not forget 3D: the sequence input is where it lives, and the per-frame MLPs deliberately use $z=0$ angles because that is what the browser serves and because the training angles had been computed with z (a 14° median gap, Section 4.2). The caveat is the signal itself: MediaPipe’s z is monocular relative depth, three ways of improving it (the library’s world landmarks, a pretrained 2-D-to-3-D lifter, bone-length correction) did not improve on the baseline they were compared with in earlier internal experiments (world landmarks 42.9% against 45.7% for the 2-D angles on photos; the lifter 10%; bone-length correction 49.7% against 54.6% for raw z on two held-out videos), and the live z from a phone camera may differ from the training videos; 3D is both the most informative and the least reliable input. A window-statistics MLP given the same (x, y, z) window does as well as the ST-GCN, which suggests that the useful part of the sequence input is the 3-D information and not the graph or the temporal convolution; we have not yet tried giving the per-frame naming model that 3-D window.

9.3 The sequence model on movement

The ST-GCN exists to model how the body moves between poses, so held-pose accuracy (Table 8, Section 8) is not the whole test. We tried to build a movement benchmark from our labels and report what the data allows.

What the labels support. In the 24 videos the cue-verified labels contain 589 holds in 21 videos and 568 consecutive hold pairs, 205 of them between different poses. Within 150 frames (6 s) there are 40 transitions between different poses, in 14 distinct directional pairs, and no pair occurs in three or more videos; within 500 frames there are 70 transitions in 33 pairs, and still none does. A named-transition benchmark with held-out videos is therefore not possible on cue-verified labels. The previous model’s 30-class vocabulary, with seven named transitions, was built from rule-defined labels instead.

A rule-defined flow task. We rebuilt that label scheme on the same videos: a window is a transition when its net angle change exceeds $15^\circ/\text{s}$ and is named by the dominant pose of its first and last third (the pose of

Table 13: Movement task (19 flow classes, rule-defined labels), previous versus current ST-GCN and the reference MLP, on the 9,458 windows of the 12 videos that arrived after the previous model was trained (clean for it). The current-architecture models are trained per fold and scored only on videos they never saw. Bold marks the best value in a column. Cobra to downward dog is the only named transition with enough support.

Model	Overall	Macro recall (19 classes)	Hold classes (16)	Cobra → down. dog recall
Previous ST-GCN (30-class checkpoint)	23.1%	14.2%	14.4%	0.0%
ST-GCN, current architecture, trained on this task	56.7%	53.1%	52.1%	78.3%
MLP on window mean/std (order-blind)	65.6%	66.7%	65.8%	87.0%

each frame is the cue-verified label where one exists and the rule engine’s pose elsewhere); otherwise it is a hold, or unrecognized. Windows with more than 15% ambiguous frames are dropped (28,544 usable windows). Keeping classes with at least 100 windows from at least three videos leaves 19 classes: 16 holds, transition:other, unrecognized and one named transition, cobra pose to downward dog (144 windows, 15 videos). So even the rule-defined scheme names a single transition on this data. The labels are defined by rules, so the task measures whether a network recovers rule-defined flow from the raw skeleton without the rules; it is not agreement with a human annotator.

Previous against current. Trained properly on cue-verified data, the current ST-GCN architecture is far better than the previous checkpoint on videos the previous one never saw: 53.1% macro recall against 14.2%, and it recognises the one named transition at 78.3% where the previous model scores 0.0%. This agrees with the held-pose result (82.1% against 18.8% macro recall, Table 8). Two cautions: the label scheme is the one the previous model was trained on, which favours the previous model rather than the current one, and the previous model’s other six named transitions have no test windows here, so its transition ability is not tested beyond cobra to downward dog. On all 24 videos (the previous model may have seen 12 of them) its macro recall is 20.9%.

ST-GCN against simpler models. The order-blind MLP is better than the ST-GCN on this task as well (66.7% against 53.1% on the clean subset). Under one recipe on all 24 videos (every fold scored on videos it never trained on), macro recall over the 19 classes is 58.6% for the MLP, 56.8% for the BiLSTM with attention, 55.6% for the temporal CNN, 53.1% for the LSTM, 48.4% for the ST-GCN without the skeleton graph and 47.7% for the ST-GCN: the ST-GCN is last, the skeleton graph again makes no measurable difference (48.4% against 47.7%) and the order-blind MLP is first. The order on the one named transition is the same (recall 68.1% for the MLP, 50.7% for the ST-GCN, 66.0% for the LSTM).

Arrow of time. To test whether any model uses frame order at all, every moving window was shown forward and time-reversed and the model had to say which. Chance is 50%, and a model that ignores order must score exactly 50%. Every model is at chance: 50.0% for the MLP (exactly, as it must be), 51.1% for the LSTM, 50.1% for the BiLSTM, 49.9% for the temporal CNN, 50.0% for the ST-GCN without the graph and 50.0% for the ST-GCN, and the training loss stays at $\ln 2 = 0.693$ for all of them except the LSTM, whose loss only begins to fall. The test is therefore uninformative: under this recipe (12 epochs, about 8,000 windows per fold) no model learns the direction of time, so it neither supports nor refutes the claim that the ST-GCN models movement.

Caveats. Rule-defined labels; 24 videos, 12 of them in the clean subset; overlapping windows (stride 12 of 60), so the effective sample is the number of holds and videos; one seed; and the ST-GCN is the slowest model to train (about 290 s per fold against 5–15 s). What this section establishes is that the current ST-GCN is much better than the previous one; it does not establish that a graph network is the best choice for movement, because we found no task, with our labels, on which it beats a window-statistics MLP.

10 Comparison with Related Work

Direct numeric comparison with other groups is not possible (we could not obtain their systems; datasets, vocabularies and splits differ), so Table 14 compares what each work reports, using only what the cited sources state; “n/r” means not reported in the sources we read, not that the feature is absent.

What we add. To the best of our reading, the works above do not report (i) a false-alarm rate on photos of poses outside the vocabulary, which is what matters when a user is in a pose the app does not know; (ii) a split

Table 14: What each work reports (qualitative; n/r = not reported in the sources we read).

Work	Task and input	Evaluation as described	Hidden joints	Feedback to the user	Runs on a phone / image stays local
Chen et al. [9]	posture recognition from silhouette star skeleton	n/r	n/r	links to training information	n/r
Yoga-82 [10]	image classification, 82 poses, 3-level labels	dataset benchmark of CNNs; motivates occlusion and view-point variety	dataset difficulty only	none (classification)	n/r
Thoutam et al. [12]	incorrect posture in uploaded videos	n/r	n/r	abnormal angles and advice	uploaded video; n/r
Mohiuddin et al. [11]	Yoga-16 (1,280 images, 16 poses); MediaPipe / YOLOv8 skeletons vs. images	96.09% on a 256-image test split of a dataset split into 896/128/256	none	none	n/r
PosePilot [13]	LSTM, BiLSTM + attention on video; new video dataset	evaluation detail n/r in the abstract	not discussed	instant corrective feedback	edge devices (design goal); n/r
Pose-to-Pose [14]	generating target poses (transitions), counterfactuals	new benchmark (PoseHPT)	n/r	not a coach	n/r
Liao et al. [15]	scoring rehabilitation exercise quality	n/r here	n/r	quality score	n/r
AsanaAI (this work)	landmark angles + 60-frame skeleton windows; cascade + ST-GCN	held-out photos and held-out videos only; false-alarm metric; controlled baselines	abstains on hidden joints; CLIFF vs. mirror measured	per-joint cues, three languages, screened, escalating	✓ tested on a budget phone / ✓ only numbers leave the device

by video or by held-out source, as opposed to a split of a single image set (Yoga-16’s test split is 256 images from the same 1,280); (iii) how hidden joints are handled, or an experiment comparing recovery methods; (iv) a deployment through a GPU driver that fails; or (v) privacy by construction, multilingual coaching with a safety screen and an offline mode with a parity proof. We also show, on our own models, how large the random-split illusion can be: the first MLP reported 91.5% validation accuracy and scored 26.8% on Commons photos it had never seen, and the first ST-GCN reported 82.6% validation accuracy but only 11.6% macro recall on held-out videos.

Where others are stronger, and what we do not do. Yoga-82 covers 82 poses with a hierarchy and thousands of images, while our guided mode covers eight poses and only three or four pass on video; PosePilot’s temporal feedback is delivered at every stage of a movement, whereas ours is per-frame cues plus a flow check; Pose-to-Pose generates how to move between poses and we do not; Liao et al. learn a quality score whereas our form score is not validated against human labels. Accuracies such as Yoga-16’s 96.09% come from a different protocol and are not comparable to ours; the controlled comparison is the one against standard methods on our own data and held-out sets (Section 8).

11 Limitations and Future Work

Lessons. (1) A random split measures memory; only held-out numbers are evidence [5]. (2) Train/serve skew hides in plain sight: raw- z training angles versus served zero- z angles (14° median gap), and a sequence model fed at 0.5 fps that was trained at 25 fps, both of which looked like “model problems” [22]. (3) Silent fallbacks are the dangerous kind: a retired language-model name returned the safe template for every request and looked like success, and a phone with a faulty GPU path crashed inside a driver instead of “having no WebGL”. (4) No method wins on every photo source, so report more than one held-out set.

Limitations.

- *No live-person accuracy yet.* All end-to-end tests use photos or recorded video; the one live test with a person had nobody in view. A study with practitioners and instructors is needed.
- *The form score is not validated against human judgement.* It is checked only by a perturbation test, it is nearly blind to arm errors (0.94→0.92 when an elbow is broken), and the learned deviation head is close to chance (12.4% vs. 6.7%); joint-level cues rest on hand-set angle bands.
- *Tree and Lunge have no angle bands,* so the app gives no per-joint feedback for them (neutral limbs;

before 2026-10-07 the near-chance learned head’s guesses were drawn, Fig. 3d). Tree is one of the eight guided poses; hand-set bands with an either-leg rule are the obvious next step.

- *Weak poses.* Mountain, cobra and child’s pose fail on the phone photo test; chair and corpse pass at only 50–60%; Seated easy failed on one live clip (named upward dog throughout); the ST-GCN reliably handles three poses.
- *Small data.* 24 videos and public photo sets; some per-pose test sets have fewer than 30 items; single-seed baselines; the held-out public photos share their sources with the training photos (exact near-duplicates are rare, Table 7, but the ranking of methods changes on wild photos, Table 6); the wild set has only 103 photos.
- *The skeleton graph is not shown to be necessary.* Under one recipe, simpler sequence models match or exceed the ST-GCN at held-pose recognition (Section 8).
- *Occlusion.* We abstain rather than recover; the pose name can still be influenced by a hallucinated hidden joint; the CLIFF study is small (141 cases, one asymmetric pose, a synthetic box, MediaPipe as reference).
- *Personalisation.* The range-of-motion profile is not user-tested and a profile learned from a poor stance could forgive real errors; the demonstration profile came from a still photo.
- *Languages.* The Hindi and Bengali interface text and cue templates were checked by a native speaker of both languages on the team; they have not had an independent review by yoga instructors or users, the language-model paraphrases are generated at run time and are not reviewed, and the forbidden-word screen is a list of English words, so it cannot flag unsafe wording in a Hindi or Bengali paraphrase; those languages rely on the reviewed templates and the escalation back-off.
- *Hardware.* Verified on one phone model and desktop browsers; other phones with the same GPU, other browsers, battery and thermal behaviour were not measured. The free-tier server can sleep, in which case basic mode applies (rules only: 36.9% accuracy and 78.3% false alarms on held-out photos).
- *Not a medical device.* We make no claim about injury prevention; no outcome study exists.

Future work. (i) A live user study in which instructors rate form, producing the labelled good/bad-form data the form head lacks; (ii) retraining the form and deviation heads with arm perturbations and with those labels; (iii) a tree–network ensemble for naming, since a random forest is the strongest method on same-source photos and the weakest on wild ones, so the two may be complementary (untested); (iv) viewpoint-robust features (3-D lifting) and a larger pose vocabulary; (v) opt-in server-side mesh recovery for hidden joints with explicit image consent, now that its benefit for legs and asymmetric poses is measured; (vi) independent review of the Hindi and Bengali coaching by instructors, and language-specific safety screens; (vii) flow scores and named transitions, the tasks on which a temporal model must prove its value over static statistics; (viii) more devices, including battery and thermal tests.

12 Conclusion

AsanaAI shows that a yoga coach can run in a browser, keep the camera image on the device and still work on a budget phone, provided evidence is held to a strict standard. Evaluated only on data never trained on, a gated cascade of residual MLPs names poses on 78.9% of held-out photos with 20.8% false alarms and seven passing poses, an ST-GCN with the right input rate reaches 78.0% macro recall on held-out videos, controlled baselines show that no frame-level method wins on every photo source, and the system degrades gracefully: it abstains on hidden joints, falls back through three pose engines, and keeps coaching offline. We also report what it cannot do yet, which is as important to a person who might follow its advice.

Availability. App: <https://yoga-posture-correction-system.vercel.app> (landing page /landing); API: <https://arko007-yoga-pose.hf.space>; code: <https://github.com/Anamitra-Sarkar/yoga-posture-correction-system>; training data and code: Hugging Face dataset Arko007/Yoga-1M (the transcripts are withheld because they are copyrighted speech).

13 Verification, Audit and Fixes

This section records what was checked before the project was written up, what was found, and what was changed. The full evidence is in `docs/REPO_AUDIT_2026-10-07.md`; none of this belongs in the research paper.

13.1 Method

All checks were read-only or black-box: tracked files, documentation, code, CI runs and deployments were inspected, and the deployed backend was queried over HTTP. Nothing was built or trained on the development machine (3.7 GB RAM). A live smoke test (`backup/verify_live.sh`) now makes seven assertions against production.

13.2 Findings that affect users or safety

#	Finding	Status
A1	Tree and Lunge have no angle bands. With the cascade on, their per-joint deviations came from the learned deviation head, which is close to chance, so a correct Tree could show a coral standing leg and a cue could name the wrong joint. Evidence: a hand-built Tree frame returned deviations of 72–73° on a straight leg; screenshot in the repository.	Fixed and live. The cascade now returns zero deviations (<code>none_no_bands</code>); the web app draws neutral limbs (<code>NO_BAND_POSES</code>); verified against production and in the shipped JavaScript.
A2	The unsafe-word screen was a list of English words, while the language-model paraphrase is live in Hindi and Bengali.	Hindi and Bengali word lists with a regression test (39 backend tests pass), reviewed by the team’s native speaker of both languages and merged into <code>main</code> ; review sheet in <code>docs/</code> . A paraphrase containing a listed word is discarded and the reviewed template is spoken instead.
A3	Code defaults were the old behaviour (cascade off, old ST-GCN); production was correct only because of Space variables.	Fixed and live. Defaults now equal the benchmarked configuration; the variables remain for roll-back.
A4	Body orientation is computed and sent but ignored while the cascade is on.	Documented.
A5	The personal profile works as designed but is unvalidated; a profile spanning 0°–180° forgives every deviation.	Documented; user study is future work.

13.3 Hygiene findings

The live smoke test pre-dated the cascade, so two of its checks passed while testing nothing; it was rewritten with realistic frames and assertions. The README overstated the safety filter and listed orientation as a pipeline input (corrected). 15 merged remote branches, each building a preview deployment, were deleted after each was verified as an ancestor of `main`; one unmerged branch was left for the owner. Other items, not changed: “Digital Twin” remains the internal name in some comments and types while the interface says “Your range of motion”; a June integration script sits outside the tests directory; a manifest shortcut opens an unused query parameter; CI does not run the backend tests; about 17 MB of old checkpoints are tracked in git; the Expo companion still carries template scaffolding and the older six-pose vocabulary.

13.4 Verified without findings

No secrets or tokens in any tracked file; 114 translation keys in each of English, Hindi and Bengali with none missing or untranslated (checked by a native speaker); all relative documentation links resolve; CI green; the Vercel production deployment equals `main`; all seven model files exist on the model repository; the owner’s saved app settings were restored after every automated screenshot run.

13.5 Calibration check

The calibration feature was exercised end to end on the deployed server (Section 6.3): a profile containing a deviating joint's angle raises the personal score, a profile that does not leaves it unchanged, and the universal score never changes. The check is repeatable with the smoke test.

13.6 A measurement worth recording

End-to-end request latency from a residential connection in India was 0.97 s at the median (p95 1.27 s, 32 requests) although server compute is 7–9 ms: the delay is the network to the hosting region. This motivates running the small MLPs on the device.

14 Repository, Operations and Reproducibility

14.1 Layout

Path	Contents
backend/	FastAPI service (also the Hugging Face Space): models, rule engine, cascade, coaching, tests, tools
frontend/	Next.js progressive web app (app at /, website at /landing) and the Capacitor Android project
mobile/	separate Expo client
modal/, planning/	data and training pipeline, experiments and the Kaggle kernels (including the baseline kernels)
docs/	benchmarks, cascade and sequence-model notes, training lessons, live-test clips, the audit
paper/	the research paper (conference format)
report/	this report
backup/	resume notes, dated copies of key documents, the live smoke test, phone-debugging tools

14.2 Deployment and operations

Pushing to main with changes under backend/ or to README.md synchronises the Hugging Face Space (GitHub Actions); Vercel builds main for production and every other branch as a preview. Since 2026-10-07 the backend defaults equal the benchmarked configuration (pose cascade on, stgcn_target_v1); rollback uses the Space variables ENABLE_POSE_CASCADE=0 and STGCN_MODEL_FILE/STGCN_ENCODER_FILE set to the previous files. The optional paraphrase needs a language-model API key stored as a Space secret. The free-tier Space sleeps when idle, in which case the browser answers in basic mode.

14.3 Testing

The backend has 70 test functions in nine files (stub models, no network); the two files touched by the 2026-10-07 changes pass (21 tests) and the safety-screen draft adds 36. The live smoke test checks the deployed system after every backend deployment. The Android workflow also type-checks the Expo companion.

14.4 Compute policy

Training and baselines run on Kaggle (CPU or GPU kernels) or Codespaces, never on the development machine; Hugging Face model files are never overwritten (new names only); kaggle kernels output is avoided and logs are read through the kernel's REST log field. The sequence-model baselines (LSTM, BiLSTM with attention, temporal CNN, window-statistics MLP, ST-GCN with and without the skeleton graph) ran on a Kaggle T4 under a second account (kernel kernel_baselines_seq_gpu_arko, token-free,

results in its log and in `results_seq_gpu_v1.json`); the first account’s weekly GPU quota had been used up.

14.5 Rebuilding the documents

`cd paper && python3 make_refs.py && pdflatex asanaai_research_paper.tex` (twice) and likewise in `report/`. IEEEtran’s bibliography style is not installed on the development machine, so `make_refs.py` builds the numbered reference list from the verified `references.bib`.

References

- [1] United Nations General Assembly, “Resolution A/RES/69/131: International Day of Yoga,” adopted 11 December 2014.
- [2] H. Cramer, R. Krucoff, and G. Dobos, “Adverse Events Associated with Yoga: A Systematic Review of Published Case Reports and Case Series,” *PLoS ONE*, vol. 8, no. 10, 2013, Art. no. e75515.
- [3] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang, and M. Grundmann, “BlazePose: On-device Real-time Body Pose Tracking,” arXiv:2006.10204, 2020.
- [4] C. Lugaresi *et al.*, “MediaPipe: A Framework for Building Perception Pipelines,” arXiv:1906.08172, 2019.
- [5] S. Kapoor and A. Narayanan, “Leakage and the Reproducibility Crisis in Machine-Learning-Based Science,” *Patterns*, vol. 4, no. 9, 2023, Art. no. 100804.
- [6] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust Speech Recognition via Large-Scale Weak Supervision,” in *Proc. 40th Int. Conf. Machine Learning (ICML)*, PMLR, vol. 202, 2023, pp. 28492–28518.
- [7] S. Yan, Y. Xiong, and D. Lin, “Spatial Temporal Graph Convolutional Networks for Skeleton-Based Action Recognition,” in *Proc. AAAI Conf. Artificial Intelligence*, vol. 32, no. 1, 2018, pp. 7444–7452, doi: 10.1609/aaai.v32i1.12328.
- [8] Z. Li, J. Liu, Z. Zhang, S. Xu, and Y. Yan, “CLIFF: Carrying Location Information in Full Frames into Human Pose and Shape Estimation,” in *European Conf. Computer Vision (ECCV)*, 2022, doi: 10.1007/978-3-031-20065-6_34.
- [9] H.-T. Chen, Y.-Z. He, C.-C. Hsu, C.-L. Chou, S.-Y. Lee, and B.-S. P. Lin, “Yoga Posture Recognition for Self-training,” in *MultiMedia Modeling (MMM)*, 2014, pp. 496–505.
- [10] M. Verma, S. Kumawat, Y. Nakashima, and S. Raman, “Yoga-82: A New Dataset for Fine-grained Classification of Human Poses,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2020, pp. 1038–1039.
- [11] M. Mohiuddin, S. M. M. Hossain, S. Khanam, P. Barua, A. Barua, and M. T. Hossain, “Integrating Skeleton Based Representations for Robust Yoga Pose Classification Using Deep Learning Models,” arXiv:2512.00572, 2025.
- [12] A. Thoutam *et al.*, “Yoga Pose Estimation and Feedback Generation Using Deep Learning,” *Computational Intelligence and Neuroscience*, 2022, doi: 10.1155/2022/4311350.
- [13] R. Gadhvi, P. Desai, and Siddharth, “PosePilot: An Edge-AI Solution for Posture Correction in Physical Exercises,” in *Iberian Conf. Pattern Recognition and Image Analysis (IbPRIA)*, 2025, arXiv:2505.19186.
- [14] B. Dittakavi, S. Maheshwari, and V. N. Balasubramanian, “Pose-to-Pose: A New Task and Benchmark for Human Pose Transition in Yoga,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2025, pp. 6138–6147.
- [15] Y. Liao, A. Vakanski, and M. Xian, “A Deep Learning Framework for Assessing Physical Rehabilitation Exercises,” *IEEE Trans. Neural Syst. Rehabil. Eng.*, vol. 28, no. 2, 2020, pp. 468–477, doi: 10.1109/TNSRE.2020.2966249.

- [16] Z. Cao, G. Hidalgo, T. Simon, S.-E. Wei, and Y. Sheikh, “OpenPose: Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 1, 2021, pp. 172–186.
- [17] K. Sun, B. Xiao, D. Liu, and J. Wang, “Deep High-Resolution Representation Learning for Human Pose Estimation,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [18] Y. Xu, J. Zhang, Q. Zhang, and D. Tao, “ViTPose: Simple Vision Transformer Baselines for Human Pose Estimation,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [19] D. Smilkov *et al.*, “TensorFlow.js: Machine Learning for the Web and Beyond,” arXiv:1901.05350, 2019.
- [20] M. Loper, N. Mahmood, J. Romero, G. Pons-Moll, and M. J. Black, “SMPL: A Skinned Multi-Person Linear Model,” *ACM Trans. Graphics*, vol. 34, no. 6, 2015, pp. 248:1–248:16, doi: 10.1145/2816795.2818013.
- [21] Y. Geifman and R. El-Yaniv, “Selective Classification for Deep Neural Networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [22] D. Sculley *et al.*, “Hidden Technical Debt in Machine Learning Systems,” in *Advances in Neural Information Processing Systems (NIPS)*, 2015, pp. 2503–2511.
- [23] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *Proc. 32nd Int. Conf. Machine Learning (ICML)*, 2015.
- [24] D. Hendrycks and K. Gimpel, “Gaussian Error Linear Units (GELUs),” arXiv:1606.08415, 2016.
- [25] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016.